

✓ ИУ5-22М Пермяков Д.К. РК2

✓ Задача классификации текстов

Необходимо решить задачу классификации текстов на основе любого выбранного Вами датасета (кроме примера, который рассматривался в лекции). Классификация может быть бинарной или многоклассовой. Целевой признак из выбранного Вами датасета может иметь любой физический смысл, примером является задача анализа тональности текста.

Необходимо сформировать два варианта векторизации признаков - на основе CountVectorizer и на основе TfidfVectorizer.

В качестве классификаторов необходимо использовать два классификатора по варианту для Вашей группы: ИУ5-22М - RandomForestClassifier, LogisticRegression

```
import pandas as pd
import time
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
# Загрузка данных
train_data = pd.read_csv('data/Corona_NLP_train.csv', encoding='latin1')
test_data = pd.read_csv('data/Corona_NLP_test.csv', encoding='latin1')
```

```
train_data.head()
```

	UserName	ScreenName	Location	TweetAt	OriginalTweet	Sentiment
0	3799	48751	London	16-03-2020	@MeNyrbie @Phil_Gahan @Chrisitv https://t.co/i...	Neutral
1	3800	48752	UK	16-03-2020	advice Talk to your neighbours family to excha...	Positive

```
train_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41157 entries, 0 to 41156
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   UserName        41157 non-null  int64
1   ScreenName      41157 non-null  int64
2   Location        32567 non-null  object
3   TweetAt        41157 non-null  object
4   OriginalTweet   41157 non-null  object
5   Sentiment       41157 non-null  object
dtypes: int64(2), object(4)
memory usage: 1.9+ MB
```

```
train_data.shape
```

```
(41157, 6)
```

```
test_data.head()
```

	UserName	ScreenName	Location	TweetAt	OriginalTweet	Sentiment
0	1	44953	NYC	02-03-2020	TRENDING: New Yorkers encounter empty supermar...	Extremely Negative
1	2	44954	Seattle, WA	02-03-2020	When I couldn't find hand sanitizer at Fred Me...	Positive

```
test_data.shape
```

```
(3798, 6)
```

```
X_train = train_data['OriginalTweet']  
y_train = train_data['Sentiment']
```

```
X_test = test_data['OriginalTweet']  
y_test = test_data['Sentiment']
```

```
def check_missing(data, name):  
    missing = data.isnull().sum()  
    print(f'Y {name} {missing} пропущенных строк')
```

```
check_missing(train_data, 'train_data')  
check_missing(test_data, 'test_data')  
check_missing(X_train, 'X_train')  
check_missing(X_test, 'X_test')  
check_missing(y_train, 'y_train')  
check_missing(y_test, 'y_test')
```

```
Y train_data UserName          0  
ScreenName          0  
Location           8590  
TweetAt            0  
OriginalTweet      0  
Sentiment           0  
dtype: int64 пропущенных строк  
Y test_data UserName          0  
ScreenName          0  
Location            834  
TweetAt             0  
OriginalTweet       0  
Sentiment            0  
dtype: int64 пропущенных строк  
Y X_train 0 пропущенных строк  
Y X_test 0 пропущенных строк  
Y y_train 0 пропущенных строк  
Y y_test 0 пропущенных строк
```

```
# Векторизация с помощью CountVectorizer
count_vect = CountVectorizer()
X_train_counts = count_vect.fit_transform(X_train)
X_test_counts = count_vect.transform(X_test)

# Векторизация с помощью TfidfVectorizer
tfidf_vect = TfidfVectorizer()
X_train_tfidf = tfidf_vect.fit_transform(X_train)
X_test_tfidf = tfidf_vect.transform(X_test)
```

```
def evaluate_model(vectorizer_name, vectorizer_train, vectorizer_test, model_name):
    start_time = time.time()
    obj_model = model
    obj_model.fit(vectorizer_train, y_train)
    predictions = obj_model.predict(vectorizer_test)

    accuracy = accuracy_score(y_test, predictions)
    duration = (time.time() - start_time) / 60

    print(f'Точность {vectorizer_name} + {model_name}: {accuracy:.4f}, время обучения: {duration:.2f} мин')

# Для CountVectorizer
evaluate_model('CountVectorizer', X_train_counts, X_test_counts, RandomForestClassifier, 'RandomForestClassifier')
evaluate_model('CountVectorizer', X_train_counts, X_test_counts, LogisticRegression, 'LogisticRegression')

# Для TfidfVectorizer
evaluate_model('TfidfVectorizer', X_train_tfidf, X_test_tfidf, RandomForestClassifier, 'RandomForestClassifier')
evaluate_model('TfidfVectorizer', X_train_tfidf, X_test_tfidf, LogisticRegression, 'LogisticRegression')
```

```
Точность CountVectorizer + RandomForestClassifier: 0.4542, время обучения: 0.01 мин
Точность CountVectorizer + LogisticRegression: 0.6087, время обучения: 0.01 мин
Точность TfidfVectorizer + RandomForestClassifier: 0.4294, время обучения: 0.01 мин
Точность TfidfVectorizer + LogisticRegression: 0.5658, время обучения: 0.01 мин
```

Лучшее качество показала модель LogisticRegression с векторизацией CountVectorizer

